

The 'sopra-standard' Package

Documentation for the 'sopra-standard' package | Version v1.0.00

2. December 2019

Florian Sihler (florian.sihler@uni-ulm.de)

1 General

1.1 Why, what for?

This $\text{\LaTeX}$ package was written as part of the Sopra in the winter semester 2019 and summer semester 2020 and served for the design of the standard document. This documentation was created together with the `sopra-base.cls` as well as the `sopra-documentation.sty` package.

The `sopra-listings` package is used to visualize the individual code excerpts. The associated package should also be embedded in this document: .

1.2 Dependencies

This package includes the following packages:

- fontawesome
- xstring
- array
- sopra-changelog

All of these packages should be part of a common $\text{\LaTeX}$ distribution, or be shipped together with this package.

1.3 Installation

The package is not delivered as a `.dtx`, which is why the following options arise:

- The package can be placed in the same directory as the document. In this case the inclusion instruction reads:

```
\usepackage{sopra-standard}
```

- The package can be placed in a subdirectory/a directory shipped with the document. In this case it is specified via the (relative) path:

```
\usepackage{./My/Path/to/sopra-standard}
```

- The package can be placed (via a symlink or similar) in one's own *texmf* tree. For example, on Linux using texlive, the package can be placed here: `~/texmf/tex/latex/`. The directory can be created and then updated using `texhash ~/texmf`. Now the package can be used like any other installed package:

```
\usepackage{sopra-standard}
```

1.4 Further peculiarities

In version v1.0.00 (`\thesosversion\rightarrowP. 3`) there are no further peculiarities.

2 Package Configuration

2.1 Accepted Parameters

In version v1.0.00 no parameters are accepted.

3 Commands and Environments

Note that the prefix `env@` only indicates the nature of an environment and is not part of the actual identifier!

The package defines underlying environments that it uses for styling. These are not presented in the documentation and should also not be used directly.

3.1 The Environments

▷ `\CreateNewStandardType{settings}{environment name}{counter name}`

Creates a new environment with the identifier `environment name`. The environment is versioned; the keys are to be passed as a comma-separated list of “key: value” pairs. As an example, here is the definition of the comment:

```
\CreateNewStandardType{symbol: \footnotesize\faCommentsO,title: Comment}
{comment}{ctrcomment}
```

The following settings are possible:

author: Sets an author; can be specified multiple times to record several.

bookmark: Sets the bookmark depth up to which the environment should be recorded.

symbol: Sets the symbol that is displayed next to the text.

title: Sets the title of the environment.

flags: Allows flags to be additionally recorded next to the box to designate metadata.

status: Allows the status for the comment or the definition to be set (for example: OK).

counter: Sets the counter to be used.

All of these settings can also be manually overridden when specifying the environment (this is recommended, for example, in the case of the author tag).

The following environments are defined: `env@filedef`, `env@comment`, `env@datadef`, `env@messagedef`, `env@cmdlinedef` and `env@begdef`.

3.2 Cross-References in Code

▷ `\CreateCodeHyperLink[Sequence]{Target}`

Requires *sopra-listings* and ensures that the “sequence” now becomes a hyperlink pointing to the marker “std:Target” set via `\label`. The `std:` is *only ever* appended automatically when no “explicit” sequence is passed. In this case the command expects a list of terms that automatically jump to markers of the same name:

```
\CreateCodeHyperLink{CharacterDescription,GenderEnum,PropertyEnum,GadgetEnum,Gadget,↔
NPCEnum,FieldStateEnum}
```

As opposed to:

```
\CreateCodeHyperLink[RequestItemChoiceMessage]{std:Message: Character and Gadget Offer}
\CreateCodeHyperLink[ItemChoiceMessage]{std:Message: Character or Gadget Selection}
```

- ▷ `\setComment{comment}`

Sets the comment that is set in the table of contents together with the next `\Chapter`^{→ p. 3} command.

- ▷ `\Chapter{identifier}`

Works like `\chapter`, but incorporates `\setComment`^{→ p. 2}:

```
\setComment{Basic information on how to read this document.}
```

```
\Chapter{General}
```

3.3 General Commands

- ▷ `\thesosversion`

Returns the current version of the package. So `\thesosversion` results in: v1.0.00

Note: via `\value{sosversion}` the version can be obtained as a 4-digit number: 1000.

A few more shortcuts are also defined, such as `\say` and `\info`, which are to be documented here soon...